Groth16 성능 최적화 상세 분석

목차

1. 성능 최적화 개요
2. 배치 다중 스칼라 곱셈 (Batch MSM)
3. 병렬 처리 최적화
4. 메모리 관리 최적화
5. FFT 최적화
6. 하드웨어별 최적화
7. 프로파일링 및 벤치마킹
8. 실제 성능 측정

성능 최적화 개요

Groth16의 성능 최적화는 여러 계층에서 이루어집니다. 각 단계별로 병목 지점을 파악하고 적절한 최적화 기법을 적용하는 것이 중요합니다.

성능 병목 지점 분석

```
// 각 단계별 시간 소모 분석 (일반적인 비율)
struct PerformanceBreakdown {
        setup_phase: f64, // ~60% (키 생성)
        proving_phase: f64, // ~35% (증명 생성)
```

```
        verification_phase: f64, // ~5% (검증)
}

// 증명 생성 단계 내부 분석
struct ProvingBreakdown {
        circuit_synthesis: f64, // ~5%
        qap_witness: f64, // ~25% (FFT 지배적)
        msm_operations: f64, // ~65% (A, B, C 계산)
        other: f64, // ~5%
}
```

최적화 전략

1. **알고리즘 최적화**: 더 효율적인 알고리즘 사용
2. **병렬 처리**: 멀티코어 CPU 활용
3. **메모리 최적화**: 캐시 효율성 및 메모리 사용량 최적화
4. **하드웨어 활용**: SIMD, GPU 등 특수 하드웨어 활용
5. **수치적 최적화**: 특수한 수학적 구조 활용

배치 다중 스칼라 곱셈 (Batch MSM)

MSM(Multi-Scalar Multiplication)은 Groth16에서 가장 시간이 많이 소모되는 연산입니다. `Σ(s_i * P_i)` 형태의 계산을 효율적으로 수행하는 것이 핵심입니다.

Pippenger 알고리즘

```
/// Pippenger 알고리즘을 사용한 MSM 구현
pub fn pippenger_msm<G: CurveGroup>(
bases: &[G::Affine],
scalars: &[BigInt],
) -> G {
    let n = bases.len();
    if n == 0 {
        return G::zero();
    }
    // 최적 윈도우 크기 계산
    let window_size = optimal_window_size(n);
    let num_windows = (256 + window_size - 1) / window_size;
    let mut result = G::zero();
    // 각 윈도우를 높은 비트부터 처리
    for window in (0..num_windows).rev() {
        // 이전 결과를 window_size만큼 시프트
        for _ in 0..window_size {
            result = result.double();
        }
        // 현재 윈도우에서 기여도 계산
        let mut buckets = vec![G::zero(); (1 << window_size) - 1];
        for (base, scalar) in bases.iter().zip(scalars.iter()) {
```

```
                let  window_val = extract_window(scalar, window, window_size);
                if  window_val != 0 {
                        buckets[window_val - 1] += base;
                }
        }
        // 버킷들을 효율적으로 합산
        let  window_sum = sum_buckets_optimized(&buckets);
                result += &window_sum;
        }
        result
}


/// 최적 윈도우 크기 계산
fn  optimal_window_size(n: usize) -> usize {
match  n {
0..=1 => 1,
2..=32 => 3,
33..=128 => 4,
129..=512 => 5,
513..=2048 => 6,
2049..=8192 => 7,
_ => 8,
}
```

```
}


/// 윈도우에서 비트 추출
fn extract_window(scalar: &BigInt, window: usize, window_size: usize) -> usize
    let start_bit = window * window_size;
    let mut result = 0usize;
    for i in 0..window_size {
        let bit_pos = start_bit + i;
        if bit_pos < 256 && scalar.get_bit(bit_pos) {
            result |= 1 << i;
        }
    }
    result
}
```

버킷 합산 최적화

```
/// 효율적인 버킷 합산 (signed binary method)
fn sum_buckets_optimized<G: CurveGroup>(buckets: &[G]) -> G {
if buckets.is_empty() {
return G::zero();
}
    let mut result = G::zero();
```

```
        let mut running_sum = G::zero();
        // 역순으로 처리하여 효율성 향상
        for bucket in buckets.iter().rev() {
            running_sum += bucket;
            result += &running_sum;
        }
        result
}
```

배치 전처리 테이블

```
/// MSM을 위한 전처리 테이블
pub struct BatchMulPreprocessing<G: CurveGroup> {
        /// 각 윈도우별 전처리된 점들
        base_table: Vec<Vec<G::Affine>>,
        /// 윈도우 크기
        window_size: usize,
        /// 기본 점
        base_point: G,
}

impl<G: CurveGroup> BatchMulPreprocessing<G> {
        pub fn new(base: G, max_scalars: usize) -> Self {
```

```
let window_size = optimal_window_size(max_scalars);
let table_size = 1 << window_size;
// 전처리 테이블 생성
let mut base_table = Vec::new();
let mut current_base = base;
for _ in 0..(256 + window_size - 1) / window_size {
    let mut window_table = vec![G::Affine::default(); tabl
    // 윈도우 테이블 계산: [0, base, 2*base, ..., (2^w-1)*base
    let mut acc = G::zero();
    for i in 0..table_size {
        window_table[i] = acc.into_affine();
        acc += &current_base;
    }
    base_table.push(window_table);
    // 다음 윈도우를 위해 base를 2^w배
    for _ in 0..window_size {
        current_base = current_base.double();
    }
}
Self {
    base_table,
    window_size,
    base_point: base,
```

```
            }
        }
        /// 배치 곱셈 수행
        pub fn batch_mul(&self, scalars: &[G::ScalarField]) -> Vec<G::Affine>
            cfg_iter!(scalars)
            .map(|scalar| self.single_mul(scalar))
            .collect()
        }
        fn single_mul(&self, scalar: &G::ScalarField) -> G::Affine {
            let scalar_bigint = scalar.into_bigint();
            let mut result = G::zero();
            for (window_idx, window_table) in self.base_table.iter().enumer
                let window_val = extract_window(&scalar_bigint, window_
                if window_val != 0 {
                    result += &window_table[window_val];
                }
            }
            result.into_affine()
        }
    }
```

MSM 성능 분석

```rust
// MSM 성능 벤치마크
struct MSMBenchmark {
        input_size: usize,
        naive_time: Duration, // O(n * 256) 이중 곱셈
        windowed_time: Duration, // 윈도우 방법
        pippenger_time: Duration, // Pippenger 알고리즘
        speedup: f64, // Pippenger vs naive
}

// 실제 측정 결과 (예시)
let  benchmarks = vec![
        MSMBenchmark {
                input_size: 1000,
                naive_time: Duration::from_millis(500),
                windowed_time: Duration::from_millis(150),
                pippenger_time: Duration::from_millis(80),
                speedup: 6.25,
        },
        MSMBenchmark {
                input_size: 10000,
                naive_time: Duration::from_millis(5000),
                windowed_time: Duration::from_millis(800),
```

```
                pippenger_time: Duration::from_millis(300),
                speedup: 16.67,
        },
];
```

병렬 처리 최적화

Rayon을 사용한 데이터 병렬성

```
use rayon::prelude::*;

/// 조건부 병렬 처리 매크로
macro_rules!  cfg_iter {
        ($e: expr) => {{
                #[cfg(feature = "parallel")]
                let  result = $e.par_iter();
                #[cfg(not(feature = "parallel"))]
                let  result = $e.iter();
                result
        }};
}

/// 병렬 제약 조건 평가
```

```
fn evaluate_constraints_parallel<F: PrimeField>(
constraints: &[Vec<(F, usize)>],
assignment: &[F],
) -> Vec<F> {
        cfg_iter!(constraints)
        .map(|constraint| {
                evaluate_constraint(constraint, assignment)
        })
        .collect()
}


/// 병렬 MSM
fn parallel_msm<G: CurveGroup>(
bases: &[G::Affine],
scalars: &[G::ScalarField],
) -> G {
        let chunk_size = (bases.len() + rayon::current_num_threads() - 1)
        / rayon::current_num_threads();
        cfg_iter!(bases.chunks(chunk_size))
        .zip(cfg_iter!(scalars.chunks(chunk_size)))
        .map(|(base_chunk, scalar_chunk)| {
        // 각 청크를 독립적으로 처리
        pippenger_msm(base_chunk, &scalar_chunk.iter()
```

```
        .map(|s| s.into_bigint())
        .collect::<Vec<_>>())
        })
        .reduce(|| G::zero(), |acc, chunk_result| acc + chunk_result)
}
```

작업 분할 전략

```
/// 동적 작업 분할
fn dynamic_work_splitting<T, F, R>(
items: &[T],
work_fn: F,
min_chunk_size: usize,
) -> Vec<R>
where
T: Sync,
F: Fn(&T) -> R + Sync,
R: Send,
{
        if items.len() < min_chunk_size {
                // 작은 작업은 순차 처리
                items.iter().map(work_fn).collect()
        } else {
```

```
                // 큰 작업은 병렬 처리
                items.par_iter().map(work_fn).collect()
        }
}

/// 적응적 병렬 처리
fn adaptive_parallel_msm<G: CurveGroup>(
bases: &[G::Affine],
scalars: &[G::ScalarField],
) -> G {
        let threshold = 100; // 경험적으로 결정된 임계값
        if bases.len() < threshold {
                // 작은 MSM은 순차 처리가 더 효율적
                sequential_msm(bases, scalars)
        } else {
                // 큰 MSM은 병렬 처리
                parallel_msm(bases, scalars)
        }
}
```

NUMA 인식 최적화

```
/// NUMA 토폴로지를 고려한 메모리 할당
```

```
fn numa_aware_allocation<T: Clone>(
data: &[T],
num_threads: usize,
) -> Vec<Vec<T>> {
    let chunk_size = (data.len() + num_threads - 1) / num_threads;
    let mut chunks = Vec::with_capacity(num_threads);
    for i in 0..num_threads {
        let start = i * chunk_size;
        let end = std::cmp::min(start + chunk_size, data.len());
        if start < end {
            // 각 스레드가 로컬 메모리에 데이터 복사
            let local_chunk = data[start..end].to_vec();
            chunks.push(local_chunk);
        }
    }
    chunks
}
```

메모리 관리 최적화

메모리 풀링

```
/// 메모리 풀을 사용한 할당 최적화
```

```rust
pub struct MemoryPool<T> {
    pool: std::sync::Mutex<Vec<Vec<T>>>,
    default_capacity: usize,
}


impl<T: Default + Clone> MemoryPool<T> {
    pub fn new(default_capacity: usize) -> Self {
        Self {
            pool: std::sync::Mutex::new(Vec::new()),
            default_capacity,
        }
    }
    pub fn get(&self) -> Vec<T> {
        let mut pool = self.pool.lock().unwrap();
        pool.pop().unwrap_or_else(|| {
            Vec::with_capacity(self.default_capacity)
        })
    }
    pub fn return_vec(&self, mut vec: Vec<T>) {
        vec.clear();
        let mut pool = self.pool.lock().unwrap();
        if pool.len() < 10 { // 풀 크기 제한
            pool.push(vec);
```

```
                }
        }
}

// 전역 메모리 풀
lazy_static::lazy_static! {
static ref FIELD_POOL: MemoryPool<ark_ff::Fp256<ark_bn254::FqParameters>> =
MemoryPool::new(10000);
static ref GROUP_POOL: MemoryPool<ark_bn254::G1Affine> =
MemoryPool::new(10000);
}
```

캐시 친화적 데이터 레이아웃

```
/// 캐시 라인 정렬된 데이터 구조
#[repr(C, align(64))] // 64바이트 캐시 라인 정렬
        pub struct AlignedConstraint<F: PrimeField> {
        pub a_terms: Vec<(F, usize)>,
        pub b_terms: Vec<(F, usize)>,
        pub c_terms: Vec<(F, usize)>,
        _padding: [u8; 0], // 컴파일러가 패딩 추가
}
```

```rust
/// 구조체 배열 vs 배열 구조체 최적화

pub struct SoAConstraints<F: PrimeField> {

        // Structure of Arrays (SoA) - 캐시 효율적

        pub a_coeffs: Vec<F>,

        pub a_indices: Vec<usize>,

        pub b_coeffs: Vec<F>,

        pub b_indices: Vec<usize>,

        pub c_coeffs: Vec<F>,

        pub c_indices: Vec<usize>,

}



impl<F: PrimeField> SoAConstraints<F> {

        /// 캐시 친화적인 제약 조건 평가

        pub fn evaluate_constraint_soa(

        &self,

        constraint_idx: usize,

        assignment: &[F],

        ) -> (F, F, F) {

                let start = constraint_idx * self.max_terms_per_constraint;

                let end = start + self.max_terms_per_constraint;

                let mut a_val = F::zero();

                let mut b_val = F::zero();

                let mut c_val = F::zero();
```

```
            // 순차적 메모리 접근으로 캐시 효율성 향상
            for i in start..end {
                if i < self.a_coeffs.len() && !self.a_coeffs[i].is_zero
                    a_val += self.a_coeffs[i] * assignment[self.a_in
                }
                if i < self.b_coeffs.len() && !self.b_coeffs[i].is_zero
                    b_val += self.b_coeffs[i] * assignment[self.b_in
                }
                if i < self.c_coeffs.len() && !self.c_coeffs[i].is_zero
                    c_val += self.c_coeffs[i] * assignment[self.c_in
                }
            }
            (a_val, b_val, c_val)
        }
}
```

메모리 프리페칭

```
/// 메모리 프리페칭을 통한 성능 향상
fn prefetch_memory<T>(data: &[T], indices: &[usize]) {
    for &idx in indices.iter().take(8) { // 8개씩 미리 로드
        if idx < data.len() {
            unsafe {
```

```
                        // x86_64 전용 프리페칭
                        #[cfg(target_arch = "x86_64")]
                        std::arch::x86_64::_mm_prefetch(
                                &data[idx] as *const _ as *const i8,
                                std::arch::x86_64::_MM_HINT_T0
                        );
                }
            }
        }
    }

    /// 프리페칭을 활용한 MSM
    fn prefetched_msm<G: CurveGroup>(
    bases: &[G::Affine],
    scalars: &[G::ScalarField],
    ) -> G {
        let mut result = G::zero();
            for (i, (base, scalar)) in bases.iter().zip(scalars.iter()).enu
                // 다음 데이터들을 미리 로드
                if i + 8 < bases.len() {
                prefetch_memory(bases, &[i + 8]);
                prefetch_memory(scalars, &[i + 8]);
            }
```

```
                result += base * scalar;

        }

        result

}
```

FFT 최적화

특수 도메인 최적화

```
/// 2의 거듭제곱 도메인에 특화된 FFT

impl<F: PrimeField> Radix2EvaluationDomain<F> {

        /// 최적화된 in-place FFT

        pub fn fft_in_place_optimized(&self, coeffs: &mut [F]) {

                let n = coeffs.len();

                assert_eq!(n, self.size());

                // 비트 역순 정렬

                self.bit_reverse_permutation(coeffs);

                // Cooley-Tukey FFT

                let mut step = 2;

                while step <= n {

                        let half_step = step / 2;

                        let w = self.group_gen.pow([(n / step) as u64]);

                        for chunk in coeffs.chunks_mut(step) {
```

```
                let mut w_pow = F::one();
                for i in 0..half_step {
                    let u = chunk[i];
                    let v = chunk[i + half_step] * w_pow;
                    chunk[i] = u + v;
                    chunk[i + half_step] = u - v;
                    w_pow *= w;
                }
            }
        step *= 2;
        }
    }
    /// SIMD를 활용한 병렬 FFT
    #[cfg(target_feature = "avx2")]
    pub fn fft_simd(&self, coeffs: &mut [F]) {
        // AVX2 명령어를 사용한 벡터화된 FFT
        // 실제 구현은 매우 복잡하므로 개념만 제시
        unsafe {
            // 8개 원소를 동시에 처리
            for chunk in coeffs.chunks_mut(8) {
                let vec = std::arch::x86_64::_mm256_load_ps(
                chunk.as_ptr() as *const f32
            );
```

```
                        // 벡터 연산 수행
                        // ...
                        }
                }
        }
}
```

혼합 기수 FFT

```
/// 혼합 기수 FFT (2, 3, 5 등의 인수 활용)
pub struct MixedRadixFFT<F: PrimeField> {
        factors: Vec<usize>,
        twiddle_factors: Vec<Vec<F>>,
}

impl<F: PrimeField> MixedRadixFFT<F> {
        pub fn new(size: usize, generator: F) -> Self {
                let factors = factorize(size);
                let twiddle_factors = Self::compute_twiddle_factors(&factors, g
                Self {
                        factors,
                        twiddle_factors,
        }
```

```
    }

        pub fn fft(&self, coeffs: &mut [F]) {
        // 각 인수에 대해 순차적으로 FFT 적용
            for (factor, twiddles) in self.factors.iter().zip(&self.twiddle
                self.radix_fft(coeffs, *factor, twiddles);
            }
        }
        fn radix_fft(&self, coeffs: &mut [F], radix: usize, twiddles: &[F]) {
            let n = coeffs.len();
            let step = n / radix;
            for start in 0..step {
                // radix-point FFT 수행
                let mut temp = vec![F::zero(); radix];
                for i in 0..radix {
                    temp[i] = coeffs[start + i * step];
                }
                // DFT 행렬 곱셈
                for i in 0..radix {
                    let mut sum = F::zero();
                    for j in 0..radix {
                        sum += temp[j] * twiddles[(i * j) % twid
                    }
                    coeffs[start + i * step] = sum;
```

```
                        }
                }
        }
}
```

NTT (Number Theoretic Transform) 최적화

```
/// 수론적 변환을 사용한 정수 도메인 FFT
pub struct NTTDomain {
        modulus: u64,
        primitive_root: u64,
        size: usize,
}

impl NTTDomain {
        /// 모듈러 산술을 사용한 빠른 NTT
        pub fn ntt_in_place(&self, coeffs: &mut [u64]) {
                let n = coeffs.len();
                // 비트 역순 정렬
                self.bit_reverse(coeffs);
                let mut step = 2;
                while step <= n {
                        let half_step = step / 2;
```

```
                let w = self.mod_pow(self.primitive_root, (self.modulus
                for chunk in coeffs.chunks_mut(step) {
                    let mut w_pow = 1u64;
                    for i in 0..half_step {
                        let u = chunk[i];
                        let v = self.mod_mul(chunk[i + half_ste
                        chunk[i] = self.mod_add(u, v);
                        chunk[i + half_step] = self.mod_sub(u, v
                        w_pow = self.mod_mul(w_pow, w);
                    }
                }
            step *= 2;
            }
        }
        #[inline]
        fn mod_mul(&self, a: u64, b: u64) -> u64 {
            ((a as u128 * b as u128) % self.modulus as u128) as u64
        }
        #[inline]
        fn mod_add(&self, a: u64, b: u64) -> u64 {
        let sum = a + b;
        if sum >= self.modulus {
            sum - self.modulus
```

```
                } else {
                        sum
                }
        }
}
```

하드웨어별 최적화

CPU 특화 최적화

```
/// CPU 기능 감지 및 최적화 경로 선택
pub struct CPUOptimizer {
        has_avx2: bool,
        has_avx512: bool,
        has_bmi2: bool,
        cache_line_size: usize,
}

impl CPUOptimizer {
pub fn new() -> Self {
        Self {
                has_avx2: is_x86_feature_detected!("avx2"),
                has_avx512: is_x86_feature_detected!("avx512f"),
```

```
                has_bmi2: is_x86_feature_detected!("bmi2"),
                cache_line_size: 64, // 일반적인 값
        }
}
pub fn optimal_msm<G: CurveGroup>(
&self,
bases: &[G::Affine],
scalars: &[G::ScalarField],
) -> G {
        match (self.has_avx512, self.has_avx2) {
                (true, _) => self.avx512_msm(bases, scalars),
                (false, true) => self.avx2_msm(bases, scalars),
                _ => self.scalar_msm(bases, scalars),
        }
}
#[cfg(target_feature = "avx512f")]
fn avx512_msm<G: CurveGroup>(
&self,
bases: &[G::Affine],
scalars: &[G::ScalarField],
) -> G {
        // AVX-512를 사용한 벡터화된 MSM
        // 16개 원소를 동시에 처리
```

```
        unimplemented!("AVX-512 MSM implementation")
}
#[cfg(target_feature = "avx2")]
fn avx2_msm<G: CurveGroup>(
&self,
bases: &[G::Affine],
scalars: &[G::ScalarField],
) -> G {
        // AVX2를 사용한 벡터화된 MSM
        // 8개 원소를 동시에 처리
        unimplemented!("AVX2 MSM implementation")
        }
}
```

GPU 가속화

```
/// CUDA를 사용한 GPU MSM (개념적 구현)
#[cfg(feature = "cuda")]
pub mod gpu_msm {
        use cudarc::driver::*;
        pub struct CudaMSM {
                device: CudaDevice,
                stream: CudaStream,
```

```
        kernel: CudaFunction,
    }
    impl CudaMSM {
        pub fn new() -> Result<Self, CudaError> {
            let device = CudaDevice::new(0)?;
            let stream = device.fork_default_stream()?;
            // CUDA 커널 로드
            let ptx = include_str!("msm_kernel.ptx");
            let module = device.load_ptx(ptx.into(), "msm", &[])?;
            let kernel = module.get_function("msm_kernel")?;
            Ok(Self {
                device,
                stream,
                kernel,
            })
        }
        pub fn msm<G: CurveGroup>(
        &self,
        bases: &[G::Affine],
        scalars: &[G::ScalarField],
        ) -> Result<G, CudaError> {
            let n = bases.len();
            // GPU 메모리 할당
```

```
                let bases_gpu = self.device.htod_copy(bases)?;
                let scalars_gpu = self.device.htod_copy(scalars)?;
                let result_gpu = self.device.alloc_zeros::<G>(1)?;
                // 커널 실행
                let grid_dim = (n + 255) / 256;
                let block_dim = 256;
                unsafe {
                    self.kernel.launch(
                    LaunchConfig {
                        grid_dim: (grid_dim as u32, 1, 1),
                        block_dim: (block_dim as u32, 1, 1),
                        shared_mem_bytes: 0,
                    },
                    (&bases_gpu, &scalars_gpu, &result_gpu, n),
                )?;
            }
            // 결과 복사
            let result = self.device.dtoh_sync_copy(&result_gpu)?;
            Ok(result[0])
            }
        }
    }
```

ARM NEON 최적화

```rust
/// ARM NEON을 사용한 벡터화
#[cfg(target_arch = "aarch64")]
pub mod neon_optimization {
use std::arch::aarch64::*;
/// NEON을 사용한 필드 연산
pub unsafe fn neon_field_add(a: &[u64], b: &[u64], result: &mut [u64]) {
        assert_eq!(a.len(), b.len());
        assert_eq!(a.len(), result.len());
        for i in (0..a.len()).step_by(2) {
                if i + 1 < a.len() {
                        let va = vld1q_u64(a.as_ptr().add(i));
                        let vb = vld1q_u64(b.as_ptr().add(i));
                        let vr = vaddq_u64(va, vb);
                        vst1q_u64(result.as_mut_ptr().add(i), vr);
                }
        }
}
/// NEON을 사용한 스칼라 곱셈
pub unsafe fn neon_scalar_mul(
scalar: u64,
points: &[ArmPoint],
```

```
    result: &mut [ArmPoint],
    ) {
        // NEON 명령어를 사용한 점 곱셈
        // 실제 구현은 타원곡선 연산의 복잡성으로 인해 매우 복잡
        unimplemented!("NEON scalar multiplication")
        }
}
```

프로파일링 및 벤치마킹

세밀한 성능 측정

```
/// 고정밀 타이머를 사용한 성능 측정
pub struct PrecisionTimer {
        start: std::time::Instant,
        measurements: Vec<(String, std::time::Duration)>,
}

impl PrecisionTimer {
        pub fn new() -> Self {
                Self {
                        start: std::time::Instant::now(),
                        measurements: Vec::new(),
```

```
            }
        }
        pub fn checkpoint(&mut self, name: &str) {
            let now = std::time::Instant::now();
            let duration = now.duration_since(self.start);
            self.measurements.push((name.to_string(), duration));
            self.start = now;
        }
        pub fn report(&self) {
            println!("Performance Report:");
            for (name, duration) in &self.measurements {
                println!(" {}: {:?}", name, duration);
            }
        }
    }

    /// 통계적 벤치마킹
    pub struct StatisticalBenchmark {
        samples: Vec<std::time::Duration>,
        warmup_iterations: usize,
        measurement_iterations: usize,
    }
```

```rust
impl StatisticalBenchmark {
    pub fn new(warmup: usize, measurements: usize) -> Self {
        Self {
            samples: Vec::with_capacity(measurements),
            warmup_iterations: warmup,
            measurement_iterations: measurements,
        }
    }
    pub fn bench<F>(&mut self, mut f: F) -> BenchmarkResult
    where
    F: FnMut(),
    {
        // 워밍업
        for _ in 0..self.warmup_iterations {
            f();
        }
        // 측정
        self.samples.clear();
        for _ in 0..self.measurement_iterations {
            let start = std::time::Instant::now();
            f();
            let duration = start.elapsed();
            self.samples.push(duration);
```

```
        }
        self.analyze()
    }
    fn analyze(&self) -> BenchmarkResult {
        let mut sorted = self.samples.clone();
        sorted.sort();
        let mean = self.samples.iter().sum::<std::time::Duration>()
        / self.samples.len() as u32;
        let median = sorted[sorted.len() / 2];
        let min = sorted[0];
        let max = sorted[sorted.len() - 1];
        // 표준편차 계산
        let variance: f64 = self.samples.iter()
        .map(|&x| {
        let diff = x.as_nanos() as f64 - mean.as_nanos() as f64;
        diff * diff
        })
        .sum::<f64>() / self.samples.len() as f64;
        let std_dev = std::time::Duration::from_nanos(variance.sqrt() a
        BenchmarkResult {
        mean,
        median,
        min,
```

```
                max,
                std_dev,
                samples: self.samples.len(),
                }
        }
}

#[derive(Debug)]
pub struct BenchmarkResult {
        pub mean: std::time::Duration,
        pub median: std::time::Duration,
        pub min: std::time::Duration,
        pub max: std::time::Duration,
        pub std_dev: std::time::Duration,
        pub samples: usize,
}
```

메모리 사용량 프로파일링

```
/// 메모리 사용량 추적
pub struct MemoryProfiler {
        peak_usage: usize,
        current_usage: usize,
```

```
        allocations: std::collections::HashMap<String, usize>,
    }

    impl MemoryProfiler {
        pub fn new() -> Self {
            Self {
                peak_usage: 0,
                current_usage: 0,
                allocations: std::collections::HashMap::new(),
            }
        }
        pub fn track_allocation(&mut self, name: &str, size: usize) {
            self.current_usage += size;
            self.peak_usage = self.peak_usage.max(self.current_usage);
            *self.allocations.entry(name.to_string()).or_insert(0) += size;
        }
        pub fn track_deallocation(&mut self, name: &str, size: usize) {
            self.current_usage = self.current_usage.saturating_sub(size);
            if let Some(allocated) = self.allocations.get_mut(name) {
                *allocated = allocated.saturating_sub(size);
            }
        }
        pub fn report(&self) {
```

```
            println!("Memory Usage Report:");
            println!(" Peak usage: {} MB", self.peak_usage / 1024 / 1024);
            println!(" Current usage: {} MB", self.current_usage / 1024 / 10
            println!(" Allocations by category:");
            for (name, size) in &self.allocations {
                if *size > 0 {
                    println!(" {}: {} MB", name, size / 1024 / 1024)
                }
            }
        }
}
```

실제 성능 측정

종합 벤치마크 결과

```
/// 실제 측정된 성능 데이터
pub struct Groth16Benchmark {
    pub curve: String,
    pub circuit_size: usize,
    pub setup_time: std::time::Duration,
    pub proving_time: std::time::Duration,
    pub verification_time: std::time::Duration,
```

```
        pub proof_size: usize,
        pub memory_usage: usize,
}

// 실제 벤치마크 결과 (예시)
pub fn benchmark_results() -> Vec<Groth16Benchmark> {
        vec![
                Groth16Benchmark {
                        curve: "BN254".to_string(),
                        circuit_size: 1000,
                        setup_time: std::time::Duration::from_millis(50),
                        proving_time: std::time::Duration::from_millis(20),
                        verification_time: std::time::Duration::from_millis(2),
                        proof_size: 128,
                        memory_usage: 10 * 1024 * 1024, // 10MB
                },
                Groth16Benchmark {
                        curve: "BLS12-381".to_string(),
                        circuit_size: 10000,
                        setup_time: std::time::Duration::from_millis(500),
                        proving_time: std::time::Duration::from_millis(150),
                        verification_time: std::time::Duration::from_millis(2),
                        proof_size: 192,
```

```
                memory_usage: 100 * 1024 * 1024, // 100MB
            },
            Groth16Benchmark {
                curve: "BLS12-377".to_string(),
                circuit_size: 100000,
                setup_time: std::time::Duration::from_millis(5000),
                proving_time: std::time::Duration::from_millis(1200),
                verification_time: std::time::Duration::from_millis(2),
                proof_size: 192,
                memory_usage: 1024 * 1024 * 1024, // 1GB
            },
        ]
}
```

최적화 효과 분석

```
/// 최적화 전후 성능 비교
pub struct OptimizationComparison {
    pub baseline: Groth16Benchmark,
    pub optimized: Groth16Benchmark,
    pub improvements: OptimizationImprovements,
}
```

```
pub struct OptimizationImprovements {
    pub setup_speedup: f64,
    pub proving_speedup: f64,
    pub verification_speedup: f64,
    pub memory_reduction: f64,
}


impl OptimizationComparison {
    pub fn analyze(baseline: Groth16Benchmark, optimized: Groth16Benchmark
        let improvements = OptimizationImprovements {
            setup_speedup: baseline.setup_time.as_nanos() as f64
            / optimized.setup_time.as_nanos() as f64,
            proving_speedup: baseline.proving_time.as_nanos() as f64
            / optimized.proving_time.as_nanos() as f64,
            verification_speedup: baseline.verification_time.as_nano
            / optimized.verification_time.as_nanos() as f64,
            memory_reduction: baseline.memory_usage as f64 / optimiz
        };
        Self {
            baseline,
            optimized,
            improvements,
        }
```

```
        }

        pub fn report(&self) {

            println!("Optimization Results:");

            println!(" Setup speedup: {:.2}x", self.improvements.setup_speed

            println!(" Proving speedup: {:.2}x", self.improvements.proving_s

            println!(" Verification speedup: {:.2}x", self.improvements.veri

            println!(" Memory reduction: {:.2}x", self.improvements.memory_r

        }

}
```

확장성 분석

```
/// 회로 크기에 따른 확장성 분석

pub fn scalability_analysis() {

        let circuit_sizes = vec![100, 1000, 10000, 100000, 1000000];

        let mut results = Vec::new();

        for size in circuit_sizes {

            let benchmark = run_benchmark_for_size(size);

            results.push(benchmark);

            println!("Circuit size: {}", size);

            println!(" Setup: {:?}", benchmark.setup_time);

            println!(" Proving: {:?}", benchmark.proving_time);

            println!(" Verification: {:?}", benchmark.verification_time);
```

```
            println!(" Memory: {} MB", benchmark.memory_usage / 1024 / 1024)
            println!();
        }
        // 복잡도 분석
        analyze_complexity(&results);
    }


fn analyze_complexity(results: &[Groth16Benchmark]) {
        println!("Complexity Analysis:");
        for i in 1..results.len() {
            let prev = &results[i-1];
            let curr = &results[i];
            let size_ratio = curr.circuit_size as f64 / prev.circuit_size a
            let setup_ratio = curr.setup_time.as_nanos() as f64 / prev.setu
            let proving_ratio = curr.proving_time.as_nanos() as f64 / prev.
            println!(" {}x size increase:", size_ratio);
            println!(" Setup time: {:.2}x", setup_ratio);
            println!(" Proving time: {:.2}x", proving_ratio);
            println!(" Complexity estimate: O(n^{:.2})",
            setup_ratio.log10() / size_ratio.log10());
        }
    }
```

이러한 성능 최적화 기법들을 통해 Groth16은 실용적인 영지식 증명 시스템으로 발전할 수 있었습니다. 특히 MSM 최적화와 병렬 처리는 가장 큰 성능 향상을 가져오는 핵심 기법들입니다. 하드웨어의 발전과 함께 더욱 정교한 최적화 기법들이 계속 개발되고 있으며, 이는 Groth16의 실용성을 더욱 높이고 있습니다.